CALENVEE.PK — FULL SYSTEM ARCHITECTURAL, SECURITY & FLOW CHAIN AUDIT REPORT

Date of Audit: October 15, 2026
Auditor: Principal Software Architect & Staff QA/Security Engineer
System Under Review: Calenvee.pk E-Commerce Platform (Web Storefront, Desktop Admin CMS, Google Sheets Backend, Google Drive Storage, Gmail Dispatch Automations)
Target Market: Pakistan (Direct-to-Consumer / Cash on Delivery & Advance Mobile Wallets)
Audit Mode: Read-Only Static Analysis & Live Endpoint Dynamic Probing

1. EXECUTIVE SUMMARY & LAUNCH-READINESS VERDICT

VERDICT: NO-GO (CRITICAL BLOCKERS REQUIRE IMMEDIATE REMEDIATION BEFORE PUBLIC LAUNCH)

Reason in 3 Lines:

- **Critical Security & PII Exposure:** The public Google Apps Script endpoint allows unauthenticated dumping of complete customer personal data (names, phone numbers, home addresses, orders) via a simple GET request.
- **Financial Exploitation Risk:** Order totals, item prices, and discounts are trusted completely from client-side payload without server-side validation or recalculation, enabling trivial price tampering to Rs. 1.
- **Concurrency & Quota Failure:** Google Apps Script lacks distributed locking (causing duplicate/colliding order IDs under concurrent checkouts) and operates on a free Gmail account that hard-caps out at 50 orders/day (100 emails/day quota).

2. COMPREHENSIVE SCORECARD (0–10)

Domain	Score	Weight	Weighted Score	Status	Primary Driver
Architecture	**5.5 / 10**	10%	0.55	⚠️ Flawed	Decoupled Git-as-CMS is fast, but lacks a true backend server; relies on client-to-Google-Apps-Script direct pipe.
Frontend	**7.5 / 10**	15%	1.125	🟢 Good	Clean React 18 + Tailwind UI, fast SPA routing, responsive mobile bottom bar; but UI/form contract contradictions exist.
Backend & API	**3.0 / 10**	15%	0.45	🔴 Critical	Zero authentication on sensitive endpoints; no rate limiting; cold starts cause timeouts; unhandled email quota crashes.
Data Layer	**4.0 / 10**	10%	0.40	🔴 Critical	Google Sheets as DB lacks ACID transactions, unique constraints, foreign keys, and concurrency locks (`LockService`).
Flow Integrity	**5.0 / 10**	20%	1.00	⚠️ Fragile	Browse-to-checkout works smoothly, but stock is NEVER deducted upon order creation, and dropped orders are lost in `localStorage` .
Integrations	**4.5 / 10**	10%	0.45	⚠️ Incomplete	Courier booking (TCS/Trax/PostEx) is 100% manual; orphaned Shopify Storefront API service; manual Drive upload setup.
Security	**2.0 / 10**	10%	0.20	🔴 Critical	Unauthenticated PII dump endpoint (CVSS 9.8); client-side price tampering; CORS `*` with zero token checks.
Performance	**6.0 / 10**	5%	0.30	⚠️ Mixed	470 KB JS bundle, but 1.2 MB uncompressed favicon and ~5 MB hero images severely slow down 3G/4G Pakistani mobile users.
DevOps & QA	**4.0 / 10**	5%	0.20	🔴 Low	Zero automated test suites (0 unit, 0 e2e); no ESLint; relies on local `exec('git push')` script on owner desktop.
Maintainability	**6.5 / 10**	0%	N/A	🟡 Moderate	Clean TypeScript definitions; clear modular file structure in frontend; monolithic 2,700-line Google Apps Script file.
OVERALL	**4.7 / 10**	**100%**	**4.675 / 10**	⛔ **NOT LAUNCH READY**	**Requires 48–72 hours of targeted hardening before public traffic.**

3. SYSTEM ARCHITECTURE & DATA FLOW MAP

```
flowchart TD
    subgraph Client_Tier ["Client Browser (Customer Device)"]
        User["Pakistani Shopper (Mobile / Desktop)"]
        SPA["Vite + React 18 SPA (calenvee.pk)"]
        LocalStore["Browser LocalStorage (Cart, Wishlist, Fallback Orders)"]
    end

    subgraph CDN_Tier ["Hosting & Edge Tier"]
        Vercel["Vercel Global Edge Network (calenvee.pk)"]
        StaticAssets["Static JS/CSS & Public Assets"]
    end

    subgraph Local_Admin_Tier ["Store Management (Desktop Server)"]
        AdminPC["Store Owner Desktop (Windows)"]
        AdminServer["Node.js admin-server.cjs (Port 4200)"]
        GitRepo["Local Git Workspace (.git)"]
        JSONData["Static CMS Files (products.json, categories.json, etc.)"]
    end

    subgraph Google_Cloud_Backend ["Backend & Persistence Tier (Google Apps Script)"]
        Webhook["Google Apps Script Web App (/exec)"]
        GSheets[("Google Spreadsheet (Orders, OrderItems, Inventory, Settings)")]
        GDrive["Google Drive Folder (Payment Proof Receipts)"]
        Gmail["Gmail API / MailApp (calenvee.pk@gmail.com)"]
    end

    subgraph ThirdParty ["Third-Party Communication & Logistics"]
        WhatsApp["WhatsApp Business Helpline (+92 300 1234567)"]
        Courier["Manual Courier Booking (TCS / Trax / PostEx Portals)"]
    end

    User <-->|HTTP / DOM| SPA
    SPA <-->|Read / Write| LocalStore
    Vercel -->|Deploys Bundle| StaticAssets
    SPA -.->|Static Fetch| StaticAssets

    SPA ==>|1. Direct Unauthenticated POST create_order| Webhook
    Webhook ==>|2. Append Rows| GSheets
    Webhook ==>|3. Save Screenshot Blob| GDrive
    Webhook ==>|4. Send Invoice & Alert| Gmail
    Gmail -->|Customer Email| User
    Gmail -->|Internal Alert| AdminPC

    AdminPC <-->|Browser UI| AdminServer
    AdminServer <-->|Read/Write JSON| JSONData
    AdminServer ==>|POST /api/publish: git commit & push| GitRepo
    GitRepo ==>|git push origin main| Vercel
    AdminServer <-->|GET ?action=get_orders (UNAUTHENTICATED)| Webhook

    User -.->|Proof Sharing & Support| WhatsApp
    AdminPC -.->|Manual CN Entry| Courier
```

4. END-TO-END FLOW CHAIN TRACING & HANDOFF MATRICES

FLOW A: Browse > Search/Filter > Product Page > Add to Cart > Cart Update

```
sequenceDiagram
    autonumber
    actor Customer as Customer
    participant UI as ProductDetailPage / Catalog
    participant Search as searchEngine.ts
    participant CartCtx as CartContext (React Context)
    participant Storage as localStorage ('calenvee_cart')

    Customer->>Search: Types query into Navbar search input
    Search->>UI: Filters PRODUCTS matching title, category, tags
    Customer->>UI: Clicks product card (e.g., /product/calenvee-obsidian-chrono-edition)
    UI->>UI: Renders specifications, color selector, size selector
    Customer->>UI: Selects Color Variant & clicks "ADD TO CART"
    UI->>CartCtx: addItem(product, selectedColor, selectedSize, quantity=1)
    CartCtx->>CartCtx: Computes unique itemId: `${product.id}-${color}-${size}`
```

```
    CartCtx->>Storage: Serializes items array to 'calenvee_cart'
    CartCtx->>UI: Updates cart icon badge, subtotal, shipping progress bar
```

Handoff Matrix — Flow A:

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Persisted	Status
A1	`Navbar.tsx` / `searchEngine.ts`	`searchInput: string`	`matchingProducts: Product[]`	Dropdown rendering	Empty array if no match; no crash	In-memory React state	**PASS**
A2	`ProductDetailPage.tsx`	`useParams<{ slug }>()`	`product: Product | undefined`	Component mount	Renders fallback "Piece no longer available" if slug not found	None	**PASS**
A3	`ProductDetailPage.tsx` (Quantity state)	`quantity: number`	`quantity` state value (default `1`)	`addItem`	`setQuantity` has NO UI control on page. Quantity is always locked to 1.	In-memory state	**PARTIAL**
A4	`CartContext.tsx:addItem`	`(product, color, size, qty)`	Updates `items: CartItem[]`	`localStorage.setItem`	If `localStorage` is full/disabled, catches error and logs warning	`localStorage.calenvee_cart`	**PASS**
A5	`CartContext.tsx` (Totals calc)	`items, appliedPromo, storeSettings`	`subtotal, discount, shippingFee, finalTotal`	Exposes via context	Pure deterministic math; handles empty cart gracefully	In-memory + storage	**PASS**

FLOW B: Checkout > Address/Phone Validation > Shipping Calculation > Payment Selection

```
sequenceDiagram
    autonumber
    actor Customer as Customer
    participant Checkout as CheckoutPage.tsx
    participant Cart as CartContext.tsx
    participant Canvas as Client Canvas (Compression)

    Customer->>Checkout: Enters Name, Mobile, Email, City, Full Address
    Customer->>Checkout: Selects Payment: COD / JazzCash / EasyPaisa / Bank Transfer
    alt Advance Payment Method Selected
        Customer->>Checkout: Selects receipt image from device camera/gallery
        Checkout->>Canvas: Downscales image & encodes as JPEG base64 (quality: 0.68)
        Canvas-->>Checkout: compressedBase64 string
    end
    Customer->>Checkout: Clicks "PLACE ORDER"
    Checkout->>Checkout: Evaluates validation rules
    alt Validation Failure
        Checkout-->>Customer: Native browser alert() popup
    else Validation Success
        Checkout->>Checkout: Opens isConfirmModalOpen confirmation dialog
    end
```

Handoff Matrix — Flow B:

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Persisted	Status
B1	`CheckoutPage.tsx:handlePlaceOrder`	`fullName, phone, address, city, email`	Validation boolean	`setIsConfirmModalOpen(true)`	Browser `alert()` halts execution	Form state in React	**BROKE** *(Contra Contra Email i "Optio line 53 151 blo checko missing*
B2	`CheckoutPage.tsx:phone` validation	`phone: string`	Only `!phone.trim()`	Next field	Allows "123", "abc", or "0000"; NO Pakistani format validation (`03XX-XXXXXXX`)	None	**BROKE** *fraudu COD o leading catastr RTO)*
B3	`CartContext.tsx:shippingFee`	`subtotal, freeShippingThreshold (Rs. 5,999)`	`shippingFee: 0 | 350`	`finalTotal`	Evaluates threshold on subtotal BEFORE coupon discount	Context state	**PASS**
B4	`CheckoutPage.tsx:handleReceiptUpload`	`e.target.files[0] (Image/PDF)`	`receiptPreview: base64`	`newOrder.receiptImage`	If PDF, skips canvas compression and reads raw data URL	Component state	**PASS**

FLOW C: Order Creation > Stock Deduction > Confirmation & Notifications

```
sequenceDiagram
    autonumber
    actor Customer as Customer
    participant Checkout as CheckoutPage.tsx
    participant Service as orderService.ts
    participant Webhook as google-apps-script.js (doPost)
    participant Sheets as Google Sheets (Orders & OrderItems)
    participant Drive as Google Drive Folder (Receipts)
    participant Gmail as GmailApp

    Customer->>Checkout: Confirms order in modal
    Checkout->>Service: submitOrderToBackend(newOrder)
    Service->>Service: Caches in localStorage.calenvee_orders
    Service->>Webhook: POST { action: 'create_order', order } (Timeout: 12s)

    critical Google Apps Script Execution
        Webhook->>Sheets: Reads LAST_ORDER_SEQ & computes next ID (NO LOCK!)
        Webhook->>Drive: Decodes base64 receipt & uploads to folder 1WFEsVs1hxJtFGLlOYMhrmJjJCcUyGTW_
        Webhook->>Sheets: Appends row to 'Orders' tab (Status: 'Placed')
        Webhook->>Sheets: Batch appends rows to 'OrderItems' tab
        Webhook--xSheets: [CRITICAL BUG] Stock is NEVER deducted in 'Inventory' tab!
        Webhook->>Gmail: sendCustomerOrderPlacedEmail(order)
        Webhook->>Gmail: sendOwnerNewOrderAlertEmail(order)
    end

    Webhook-->>Service: JSON { success: true, orderId: 'CAL-ORD-1000012' }
    Service-->>Checkout: OrderSubmissionResult
    Checkout->>Checkout: Clears Cart, triggers Confetti, renders Order Completed screen
```

Handoff Matrix — Flow C:

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Pe
C1	`orderService.ts:submitOrderToBackend`	`order: OrderDetails`	`Promise<OrderSubmissionResult>`	Webhook HTTP POST	Caches in `localStorage.calenvee_orders` before network call	`localSt`

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Pe
C2	Webhook Concurrency Lock	Concurrent HTTP requests	Atomically locked order creation	Sequential ID generation	**Zero LockService used.** Two concurrent checkouts calculate the identical Order ID!	None
C3	Webhook Price Recalculation	`order.total, order.items`	Verified total in PKR	Write to Sheet	**Zero verification.** Accepts client-passed `order.total` blindly.	`Orders!`
C4	Stock Deduction	`order.items: CartItem[]`	Decrement `stockCount`	Inventory update	**Zero logic exists.** Neither Google Sheets `Inventory` nor `products.json` stock is decremented.	None
C5	Google Drive Receipt Upload	`order.receiptImage (base64)`	`receiptUrl: string`	Saved in Col Q & hyperlinked	Falls back to "Calenvee Payment Receipts" root folder if ID inaccessible	Google D
C6	Customer Invoice Email	`order.email, order`	HTML MIME email via Gmail	Gmail delivery	Free Gmail quota limit (100 emails/day) hard-stops script when exceeded	Gmail Se
C7	Client Network Drop / Timeout	Webhook aborts after 12s	Fallback `{ success: true, source: 'local_fallback' }`	Checkout renders success screen	Order is stranded in customer's browser `localStorage`; merchant NEVER gets it!	Custome

---

FLOW D: Admin Order Reception > Status Updates > Courier Dispatch

```
sequenceDiagram
    autonumber
    actor Admin as Store Management
    participant Sheet as Google Sheets UI
    participant Script as google-apps-script.js
    participant Customer as Customer Email / WhatsApp

    Admin->>Sheet: Opens Orders tab, reviews verification call
    Admin->>Sheet: Manually inputs Courier Name (Col K) & Tracking ID (Col L)
    Admin->>Sheet: Selects row & clicks "Calenvee Store > 4. Send Dispatch Emails"
    Script->>Script: Reads selected row data & validates tracking number
    Script->>Customer: Dispatches luxury HTML email with courier name and tracking code
    Script->>Sheet: Marks Dispatch Email Sent = 'Yes' (Col P)
```

Handoff Matrix — Flow D:

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Persisted	Status
D1	`google-apps-script.js:batchSendCustomerDispatchEmail`	Active Sheet selection	Dispatches email to customer	Updates Col P to 'Yes'	Skips rows with no email or already dispatched; alerts via UI pop-up	Google Sheets Col P	**PASS**
D2	Courier API Booking (TCS/Trax/PostEx)	Order payload	Consignment number (CN)	Automated label generation	**No API integration exists.** Merchant must manually log into courier portal, generate CN, and paste it into Sheet.	Courier external portal	**UNVERIFIED / MANUAL**

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Persisted	Status
D3	`admin-server.cjs:/api/orders/update-status`	`{ orderId, status, courier, trackingNumber }`	Updates local cache & cloud	Updates Google Sheet via webhook	If webhook returns error, local cache is updated anyway (drift risk)	`ordersCache.json`	**PARTIAL**

FLOW E: Delivery > COD Reconciliation > Returns & Cancellations

```
sequenceDiagram
    autonumber
    actor Customer as Customer
    actor Admin as Store Management
    participant Sheet as Google Sheets UI
    participant Script as google-apps-script.js (handleCancelOrder)
    participant Gmail as GmailApp

    Customer->>Admin: WhatsApp message requesting cancellation / return
    Admin->>Sheet: Selects order row & clicks "Calenvee Store > 2. Send Cancellation Email"
    Sheet->>Admin: Prompt: "Enter Cancellation Reason"
    Admin->>Sheet: Enters reason (e.g., Customer rejected on verification call)
    Script->>Sheet: Sets Order Status = 'Cancelled' (Col J)
    Script->>Gmail: Generates luxury cancellation email with reason & advance refund notice
    Gmail-->>Customer: Customer receives cancellation notice
```

Handoff Matrix — Flow E:

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Persisted	Status
E1	`google-apps-script.js:handleCancelOrder`	Selected row in Sheet	Status updated to 'Cancelled', cancellation email sent	Customer notified	If cancelled order was advance payment, highlights 24-48h refund notice in email	Google Sheets Col J	**PASS**
E2	COD Rider Cash Reconciliation	Courier remittance invoice	Reconciled revenue vs booked	Bank deposit	**Zero automated reconciliation.** Relies entirely on manual accountant entry.	Manual	**MANUAL**
E3	Return to Origin (RTO) Stock Restock	Returned parcel	Restocked item count	Inventory update	Manual entry into Google Sheets.	None	**MANUAL**

FLOW F: Customer Authentication & Session Management

Handoff Matrix — Flow F:

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Persisted	Status
F1	Signup / Login / Password Reset	N/A	N/A	N/A	**Does not exist.** Store operates exclusively as 100% Guest Checkout.	None	**N/A (By Design)**
F2	Order History Lookup	Customer identifier	Previous orders list	Customer Portal	**Does not exist on web.** Customers must check their email or contact WhatsApp support to view past orders.	`localStorage.calenvee_orders` (Client only)	**PARTIAL**

FLOW G: Admin Panel — Product, Inventory, Price & Content Management

```
sequenceDiagram
    autonumber
    actor Admin as Store Owner
    participant WebUI as localhost:4200 (Browser)
    participant Server as admin-server.cjs
    participant JSON as products.json / storeSettings.json
```

```
    participant Cloud as google-apps-script.js
    participant Git as Git Repo & Vercel

    Admin->>WebUI: Edits product price, title, or image on localhost:4200
    WebUI->>Server: POST /api/products/save
    Server->>JSON: Overwrites src/data/products.json
    Server->>Cloud: POST action: 'sync_catalog' to Google Apps Script
    Cloud->>Cloud: Synchronizes 25 products to Google Sheets 'Inventory' tab
    Admin->>WebUI: Clicks "Publish Live" button
    Server->>Git: Executes: git add -A && git commit -m "..." && git push origin main
    Git->>Git: Pushes commit to GitHub
    Git-->>Vercel: GitHub Webhook triggers Vercel production rebuild
    Vercel-->>WebUI: New static build deployed to https://calenvee.pk
```

Handoff Matrix — Flow G:

Step	Function / Component	Input Contract	Output Contract	Next Step	Failure Behavior	State Persisted	Status
G1	`admin-server.cjs:/api/products/save`	`Product` JSON object	Updated `products.json`	Calls `syncToCloud('sync_catalog')`	Reverts if file system write fails	`src/data/products.json`	**PASS**
G2	`admin-server.cjs:syncToCloud`	`action, data, retries=1`	Google Webhook response	Cloud sync badge update	Retries once on 404/500 with 30s timeout; queues requests sequentially	Google Sheets Inventory tab	**PASS**
G3	`admin-server.cjs:/api/publish`	POST trigger	`git commit` & `git push`	Vercel rebuild trigger	If git push fails (e.g. merge conflict or no internet), returns detailed error log	GitHub repository	**PASS**
G4	`admin-server.cjs:Auth`	Any HTTP request to port 4200	Full admin access	Admin dashboard	**Zero authentication.** Anyone on local network or via DNS rebinding can control store.	None	**BROKEN**

5. FINDINGS TABLE (SORTED BY SEVERITY)

ID	Severity	Title	Location	Evidence	Business Impact
SEC-01	**CRITICAL**	**Unauthenticated Public PII Data Dump via Webhook**	`backend/google-apps-script.js:1347-1365`	Probed live: `Invoke-RestMethod "https://script.google.com/macros/s/.../exec?action=get_orders"` returned full customer database with names, phone numbers, addresses, and order history without any token.	Severe legal, regulatory (Pakistan PECA Act), and reputational catastrophe. Competitors or scrapers can harvest all customer records.
SEC-02	**CRITICAL**	**Client-Side Price & Discount Tampering**	`backend/google-apps-script.js:920-939` & `src/pages/CheckoutPage.tsx:184-187`	`handleCreateOrder` extracts `order.total` directly from incoming POST JSON and commits it to Google Sheets without re-calculating product prices from server-side catalog.	Customers can alter payload to buy Rs. 5,000 watches for Rs. 1. COD parcels will be dispatched at fraudulent prices.
DATA-01	**CRITICAL**	**No Concurrency Lock on Sequential Order IDs**	`backend/google-apps-script.js:828-875`	`getNextSequentialOrderId` reads `ScriptProperties` and sheet rows without acquiring `LockService.getScriptLock()`. Code contains 0 references to `LockService`.	Simultaneous checkouts generate identical Order IDs (e.g. two orders both named `CAL-ORD-1000015`), overwriting rows or corrupting accounting.

ID	Severity	Title	Location	Evidence	Business Impact
DATA-02	**CRITICAL**	**Zero Inventory Stock Deduction Upon Order Placement**	`backend/google-apps-script.js:910-975`	Probed code: `handleCreateOrder` appends to `Orders` and `OrderItems`, but NEVER modifies `Inventory` sheet stock or flags out-of-stock items.	Massive overselling risk during marketing campaigns/sales drops. Multiple customers purchase non-existent inventory.
REL-01	**HIGH**	**Silent Order Loss on Client Webhook Timeout**	`src/services/orderService.ts:38-68` & `src/pages/CheckoutPage.tsx:195-214`	When webhook times out (12s), `orderService` returns `{ success: true, source: 'local_fallback' }`. Checkout clears cart and shows "Order Placed!", but order is trapped in customer's `localStorage` and never reaches merchant.	Customer thinks order was placed and waits at home; store owner never sees it. Unfulfilled orders, angry customers, lost revenue.
REL-02	**HIGH**	**Free Gmail Daily Quota Exhaustion (100 Emails/Day)**	`backend/google-apps-script.js:1573, 1588, 1604, 2264`	Order placement triggers 2 emails (1 to customer, 1 to owner). Account is `calenvee.pk@gmail.com` (free consumer account). Free Gmail quota is strictly 100 emails/day.	At 50 orders/day, email quota hits 100%. `GmailApp.sendEmail` crashes with fatal exception. All subsequent checkouts fail or drop notification emails.
UX-01	**HIGH**	**Checkout Form UI Contract Contradiction on Email**	`src/pages/CheckoutPage.tsx:151-154` vs `src/pages/CheckoutPage.tsx:532`	Line 532 displays label: `"Email Address (Optional)"`. But line 151 triggers an `alert()` and BLOCKS submission if email is empty or lacks `@`.	High cart abandonment rate! Customers who don't want to provide an email see "Optional", click Place Order, get blocked by an alert, and abandon.
UX-02	**HIGH**	**Missing Pakistani Phone Number Regex Validation**	`src/pages/CheckoutPage.tsx:141`	Validation only checks `!phone.trim()`. Probed behavior: accepts `"12345"`, `"abc"`, or `"00000000000"`.	Fake/garbage phone numbers submitted by pranksters or bots. Couriers cannot deliver COD without a working phone number. Massive RTO shipping fee penalties.
SEO-01	**MEDIUM**	**Broken & Mismatched URLs in Sitemap.xml**	`public/sitemap.xml:19-32` vs `src/App.tsx:61-78`	`sitemap.xml` lists `/track-order`, `/bestsellers`, `/terms`. In `App.tsx`, these routes do NOT exist (`/bestsellers` is `/best-sellers`, `/terms` is `/terms-and-conditions`, `/track-order` is missing).	Googlebot encounters soft 404 redirects to homepage. Harms SEO ranking and Google Search Console indexation.
PERF-01	**MEDIUM**	**Bloated Static Assets in Public Directory**	`public/favicon.png` & `public/hero/`	`public/favicon.png` is **1,173 KB (1.17 MB)**! `public/hero/hero-watch-gold.png` is **995 KB**. Total hero images exceed **5 MB**.	Severe initial page load delay on Pakistani mobile networks (Jazz/Zong 4G). High bounce rate on mobile.
SEC-03	**MEDIUM**	**Unprotected Desktop Admin Server (Port 4200)**	`scripts/admin-server.cjs:3390-3408`	`server.listen(4200)` has no password, no Basic Auth, and responds to all origins with `Access-Control-Allow-Origin: *`.	Any malicious website visited by the store owner in another tab could potentially issue cross-origin requests to `http://localhost:4200/api/publish` or modify products.
CODE-01	**LOW**	**Orphaned Shopify API Service**	`src/services/shopify.ts:1-168`	Code defines `getProducts`, `shopifyFetch`, `createShopifyCheckout`. Verified: NOT imported or called by ANY component in the entire project.	Code bloat; confusing architecture for incoming developers.
DEV-01	**LOW**	**Vulnerable NPM Dependencies**	`package.json` / `npm audit`	`npm audit` reported 7 vulnerabilities (5 High: `braces` via `tailwindcss/chokidar`, 2 Moderate: `react-router` open redirect CVE-2025-68470).	Potential DoS via malicious regex in build tools; open redirect in client-side router navigation.

6. BROKEN-CHAIN & ORPHAN AUDIT

Broken Chains (Where step N fails to hand over to step N+1):

- **Order Creation ➔ Inventory Deduction:**
  - Step: Customer orders product (e.g., 2 units of Obsidian Chrono Watch).
  - Expected: `Inventory` tab stock count decreases by 2.
  - Reality: Hand-off does not exist. `handleCreateOrder` finishes without touching the `Inventory` tab.
- **Offline Fallback ➔ Merchant Notification:**

- Step: Customer checkout encounters webhook timeout or cold start abort (> 12s).
- Expected: System queues the order and retries in the background until merchant receives it.
- Reality: Hand-off breaks. Order is dumped into `localStorage` and marked as "success" on screen. The merchant never receives the order.

- **UI Email Requirement ➔ Form Validation:**
  - Step: Customer reads "Email Address (Optional)" and leaves field blank.
  - Expected: Form submits successfully without email, sending invoice via WhatsApp instead.
  - Reality: Validation interceptor abruptly calls `alert(...)` and stops execution.

- **Sitemap ➔ Route Resolution:**
  - Step: Search crawler requests `https://calenvee.pk/track-order`.
  - Expected: Renders Order Tracking page.
  - Reality: Hand-off breaks. Route does not exist; falls through to `HomePage.tsx`.

Orphaned / Dead Assets:

- `src/services/shopify.ts`: 168 lines of dead Shopify GraphQL code. Not imported by any file.
- `ProductDetailPage.tsx:46` (`setQuantity`): State setter is initialized but has no bound button, slider, or input in the JSX.
- `public/invoice-preview.html`: Static standalone file (19.7 KB) not linked from navigation or router.

---

7. PAKISTAN MARKET READINESS & COD SPECIFICS

- **Cash on Delivery (COD) Fraud & RTO Vulnerability:**
  - In Pakistan, unverified COD orders suffer a **25%–35% Return to Origin (RTO)** rate if phone numbers are not strictly verified.
  - Because Calenvee allows any text in the phone input and generates an automated order without OTP/SMS verification, prank orders will cause direct courier shipping charge losses (couriers charge Rs. 250–350 return shipping even if parcel is rejected).
  - **Recommendation:** Implement mandatory 11-digit mobile validation (`03XXXXXXXXX`), and utilize the WhatsApp confirmation call step (already indicated in dashboard) before moving status from "Placed" to "Confirmed".

- **WhatsApp Reliance:**
  - The integration with WhatsApp (`https://wa.me/923001234567`) is present in the header, footer, product page, order completed screen, and returns page. This aligns well with Pakistani consumer habits.
  - **Observation:** Ensure the actual active WhatsApp business number is configured in `src/data/storeSettings.json` before launch (currently placeholder `+923001234567`).

---

8. PRIORITIZED ACTION ROADMAP

Phase 1: Pre-Launch Blockers (Must Fix in Next 48 Hours)

- [ ] **BLOCKER 1 (Security):** Add a shared authentication secret key between the Frontend/Admin server and Google Apps Script to lock down `get_orders`, `sync_settings`, and `update_order_status`. Deny public access to customer PII.
- [ ] **BLOCKER 2 (Security):** Implement server-side order total recalculation in `backend/google-apps-script.js` so client cannot tamper with prices.
- [ ] **BLOCKER 3 (Data Integrity):** Add `LockService.getScriptLock()` in Google Apps Script `handleCreateOrder` to prevent duplicate Order IDs under concurrent traffic.
- [ ] **BLOCKER 4 (UX / Conversion):** Fix the email label mismatch in `CheckoutPage.tsx:532` (change "Optional" to "Required for Invoice" OR remove blocking check on line 151).
- [ ] **BLOCKER 5 (Fraud Prevention):** Add Pakistani 11-digit cellular regex validation (`/^(03\d{9})$/`) on the checkout phone input.
- [ ] **BLOCKER 6 (Performance):** Downscale `public/favicon.png` from 1.17 MB to < 20 KB.

Phase 2: Week 1 Post-Launch Hardening

- [ ] **Email Quota Upgrade:** Migrate from `@gmail.com` to Google Workspace OR configure a transactional email API (Resend / SendGrid) to eliminate the 100 emails/day cap.
- [ ] **Inventory Linking:** Add stock decrement logic to `handleCreateOrder` in Google Apps Script.
- [ ] **Client Resiliency:** In `orderService.ts`, if webhook fails, provide an immediate fallback UI: "Network hiccup — click here to instantly send order slip via WhatsApp" rather than showing false success.
- [ ] **Fix Sitemap:** Update `public/sitemap.xml` to match the exact routes in `src/App.tsx`.
- [ ] **SEO & Meta Pixel:** Insert Meta Pixel (Facebook Pixel) and GA4 scripts in `index.html` to track `PageView`, `ViewContent`, `AddToCart`, and `Purchase` events.

Phase 3: Month 1 Scalability & Logistics

- [ ] **Automated Courier API Integration:** Replace manual tracking number entry with automated booking via TCS / Trax / PostEx Open APIs.

- ☐ **Customer Order Tracking Page:** Create a real `/track-order` route allowing customers to check parcel status using their phone number or Order ID.
- ☐ **Automated Tests:** Set up Vitest + React Testing Library to test checkout math, coupon logic, and cart state automatically on CI.

---

9. WHAT WAS NOT VERIFIED & WHY

- **Live Courier Booking API Responses:** Marked **MANUAL / NOT VERIFIED** because the system has no code integrations for courier APIs; bookings are conducted manually by staff outside the codebase.
- **Actual Google Drive Folder Quota:** Probed folder ID `1WFEsVs1hxJtFGLlOYMhrmJjJCcUyGTW_` via code analysis; verified code structure and permissions diagnostic function, but actual Google Drive storage capacity (15 GB limit) was not queried.
- **Simulated High-Volume Load Test (> 50 Concurrent Users):** Not executed against live Google Apps Script endpoint to avoid exhausting the client's live daily email and Apps Script execution quotas prior to launch.